
Gradient Descent

— Dr. Anuradha Yenikar —

Syllabus Points -

Gradient Descent (GD)

Vanishing Gradient Problem

Momentum Based GD

Nesterov Accelerated GD

Stochastic GD

AdaGrad

RMSProp

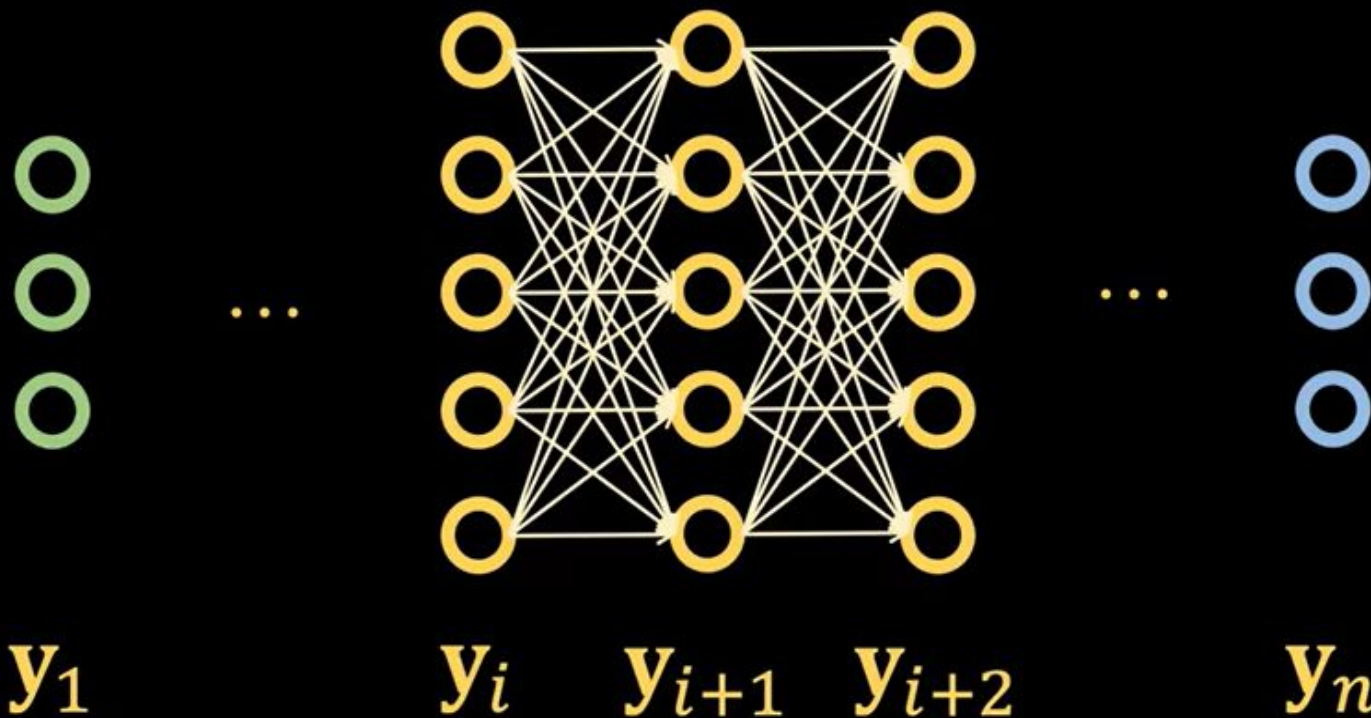
Gradient Descent Animation of Best Fit Line using Matplotlib

https://youtu.be/_JCQAJHMFug?feature=shared

Neural network outputs

Loss/Error Function

$$L(\mathbf{y}_n)$$



Loss function

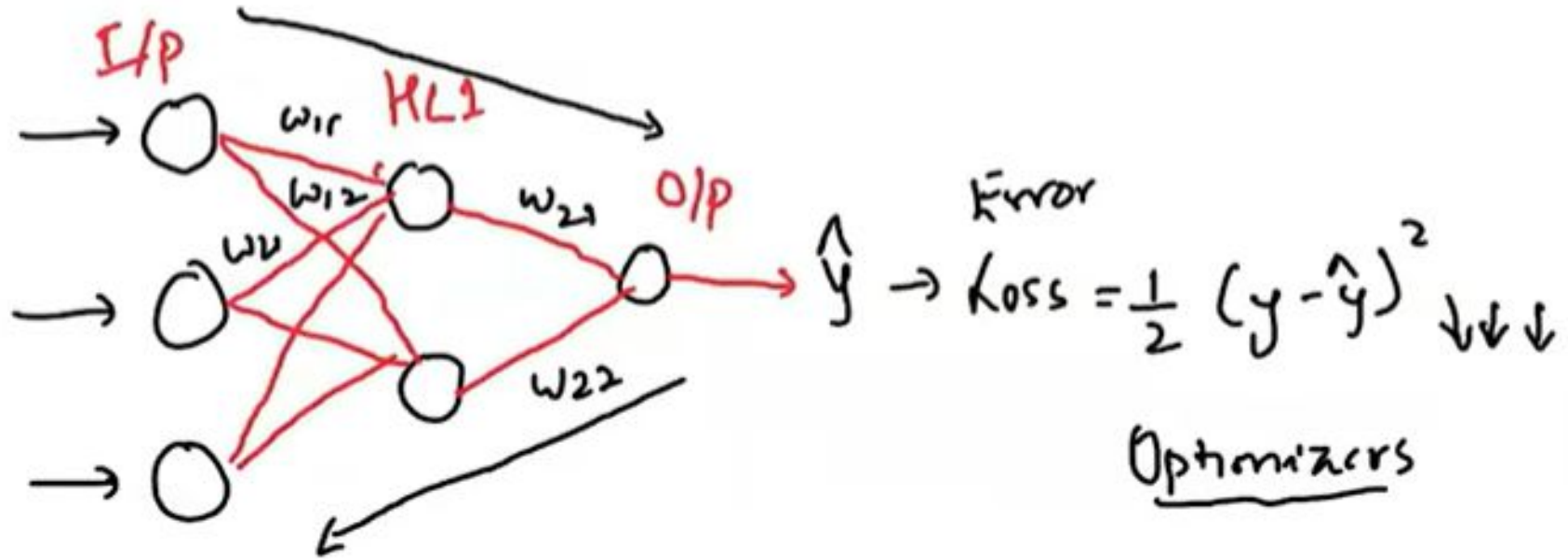
Loss function: (Predicted output — actual output) is nothing but loss and this loss we pass inside optimizer.

Role of optimizer : It is responsible for reducing the losses and to provide the most accurate results possible by calculating the attributes of the neural network.

Optimizer

- It is nothing but an algorithm or methods used to change the attributes of the neural networks such as weights and learning rate in order to reduce the losses.
- In other words, how we should change the weights and learning rates of our neural network to reduce the loss

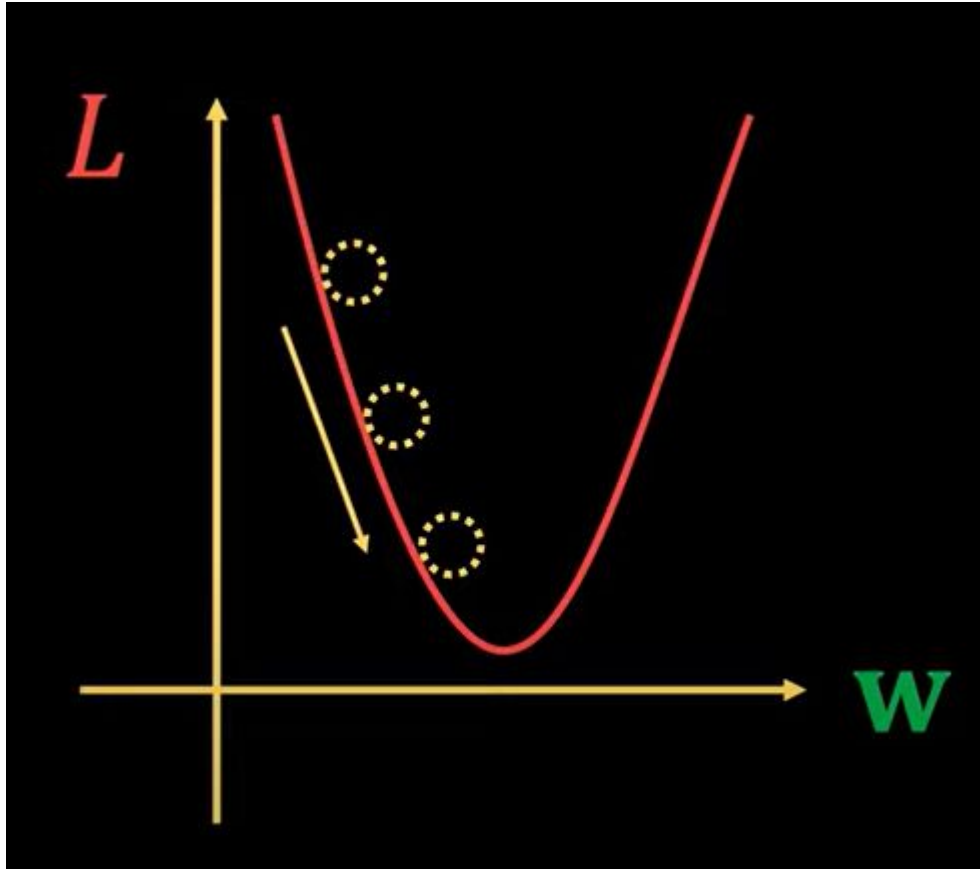
Optimizer - Gradient Descent



$$w_{\text{new}} = w_{\text{old}} - \eta \frac{\partial L}{\partial w_{\text{old}}}$$

→ GRADIENT DESCENT

Gradient Descent



Loss function

Predicted output

Input

$$L(y, \hat{y}) = L(x, \hat{y}, \mathbf{w})$$

Ground truth output

Network weights/parameters
(vector)

Loss function

$$\text{loss} = \frac{1}{2} \sum_{i=1}^n (y_i - \hat{y}_i)^2$$

area	bedrooms	price
2600	3	550000
3000	4	565000
3200	3	610000
3600	3	595000
4000	5	760000
4100	6	810000

6 samples

$$\widehat{\text{price}} = 3005$$

$$\text{price} = 565000$$

Machine learning model

$$\text{price} = w1 * \text{area} + w2 * \text{bedrooms} + \text{bias}$$

$$\text{price} = 1 * \text{area} + 1 * \text{bedrooms} + 1$$

$$\text{error2} = (\text{price} - \widehat{\text{price}})^2$$

$$\text{error6} = (\text{price} - \widehat{\text{price}})^2$$

$$\text{Total Error} = \text{error1} + \text{error2} + \dots + \text{error6}$$

$$\text{Mean Squared Error (a.k.a. MSE)} = \frac{\text{Total Error}}{6}$$

$$w1 = w1 - \text{learning rate} * \frac{\partial(\text{MSE})}{\partial w1}$$

$$w1 = 1 - (-50) = 51$$

$$w2 = w2 - \text{learning rate} * \frac{\partial(\text{MSE})}{\partial w2}$$

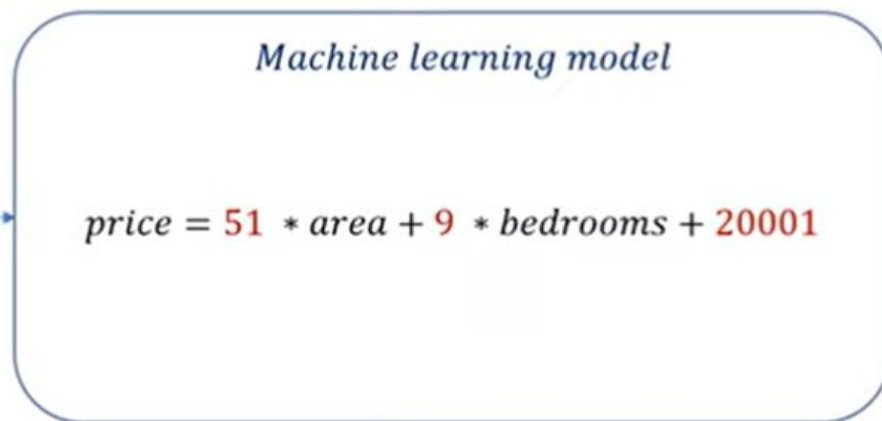
$$w2 = 1 - (-8) = 9$$

$$b = b - \text{learning rate} * \frac{\partial(\text{MSE})}{\partial b}$$

$$\text{bias} = 1 - (-20000) = 20001$$

End of first Epoch

area	bedrooms	price
2600	3	550000
3000	4	565000
3200	3	610000
3600	3	595000
4000	5	760000
4100	6	810000



$$\widehat{\text{price}} = 229155$$

$$\text{price} = 550000$$

$$\text{error}_6 = (\text{price} - \widehat{\text{price}})^2$$

End of second epoch

area	bedrooms	price
2600	3	550000
3000	4	565000
3200	3	610000
3600	3	595000
...	...	...
4100	6	810000

10 million samples

- 1) To find cumulative error for first round (epoch) now we need to do a forward pass for **10 million samples**
- 2) We have **2 features** (area and bedroom). This requires finding **20 million derivatives**

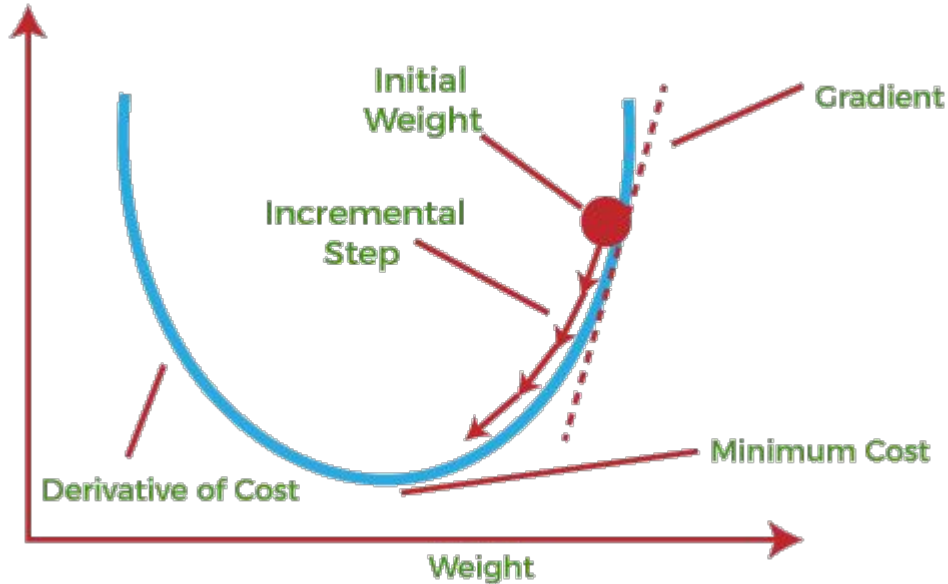


WHAT WE JUST
LOOKED AT IS CALLED

BATCH
GRADIENT DESCENT

Gradient Descent

- If we move towards a negative gradient or away from the gradient of the function at the current point, it will give the **local minimum** of that function.
- Whenever we move towards a positive gradient or towards the gradient of the function at the current point, we will get the **local maximum** of that function.



Batching

$$L = L(\mathbf{x}, \mathbf{w})$$

$$\mathbf{x} = [x_1, x_2, \dots, x_m]$$

$$L(\mathbf{x}, \mathbf{w}) = \frac{1}{m} \sum_{i=1}^m L(x_i, \mathbf{w})$$

Drawback of Batch Gradient Descent

1. Require huge size of resource (i.e. RAM to process complex and large size data)
2. Computationally expensive
3. Time consuming

Types of Optimizer

There are different types of optimizer :

- Batch Gradient descent
- Mini batch Gradient descent
- Stochastic Gradient descent

These types are nothing but different approach to send a data inside a network.

Batch Gradient Descent

BGD is a variation of the gradient descent algorithm that calculates the error for each eg in the training datasets, but only updates the model after all training examples have been evaluated.

Advantages:

It is more computationally efficient.

It is a learnable parameter : whenever we are trying to calculate a new weight, we are trying to consider all the data which is available to us based on the summation of the loss. So, we are trying to find out or derive the new value of the weight / bias , which is a learnable parameter.

Advice: try to decrease your batch size.

Disadvantages:

Memory consumption is too high: we are trying to send all the data inside the network one by one, so, we need some kind of memory to store a loss which we have received in each and every iterations. Once we are done with passing datasets through the network, we calculate the loss. So this this case , memory consumption will be too high, and this happens in each and every step.

If memory consumption is too high, we can say that the computation will be high and calculation will be very slow and so the optimization will be slower as compared to any other optimizer.

Stochastic Gradient descent (SGD)

Learning Rate

$$\mathbf{w}_{t+1} = \mathbf{w}_t - \eta \mathbf{g}(\mathbf{w}_t)$$

Update rule

Here One record at a time taken, resource required is less
but convergence will be very slow.

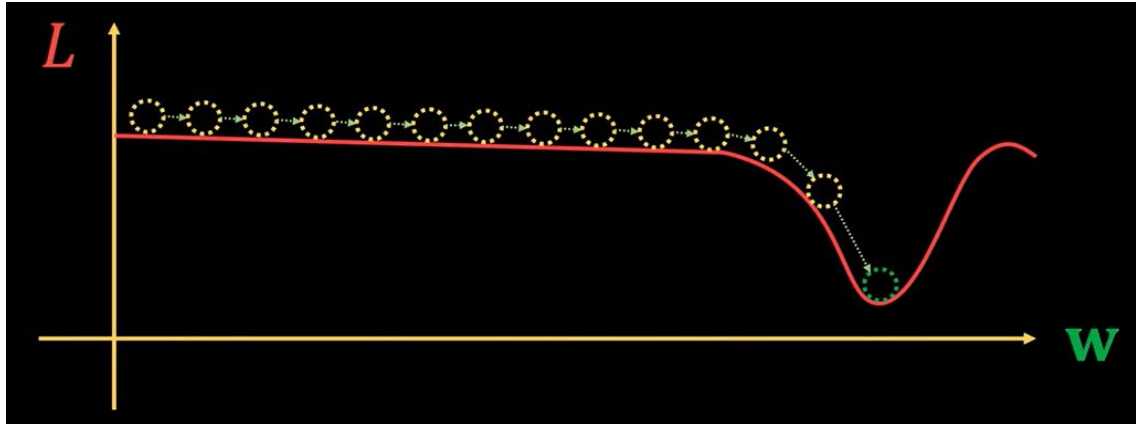
Suppose you have 10k records data

1. In 1 epoch, 10k iterations
2. In 20 epoch, $20 \times 10k = 2L$ iterations

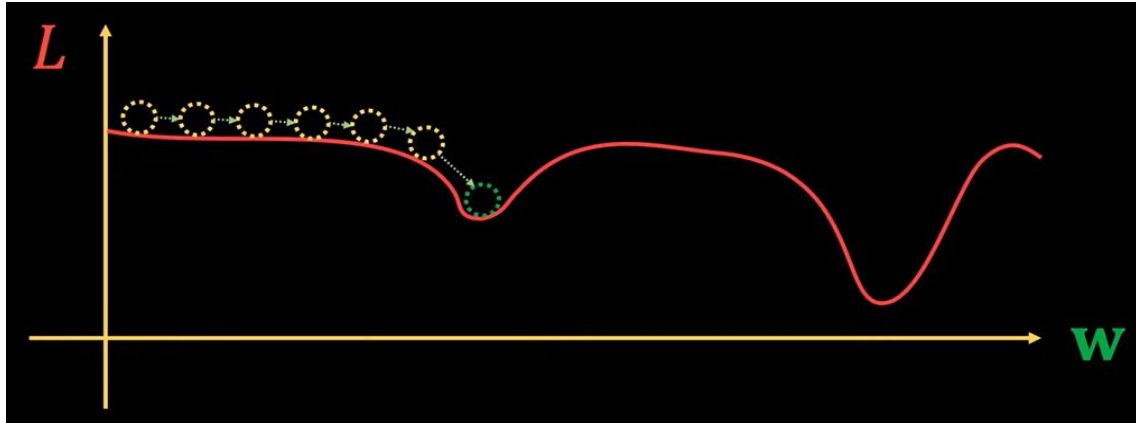
Convergence =

Reaching a point in which gradient descent makes very small changes in your objective function is called convergence, which doesn't mean it reached the optimal result

Drawbacks of SGD



← Low Learning Rate



← Local Minima

Mini batch SGD

For every epoch, there will be a batch size

Suppose,

- Data size = 10k records
- **Batch size is 1000,**
- Epochs = 20

For 1 Epoch = 10k / 1000, iterations = 10

Epoch = How many number of times we are training the model.

$$\boxed{\text{Loss} = \frac{1}{2} \sum_{i=1}^t (y_i - \hat{y}_i)^2}$$

t = batch size

Advantage: Less resources, less computational expensive

Mini batch SGD

Epoch 1

Iteration 1 : 1000 batch

----> Forward Propagation

<---- Back Propagation

Iteration 2 : 1000 batch

----> Forward Propagation

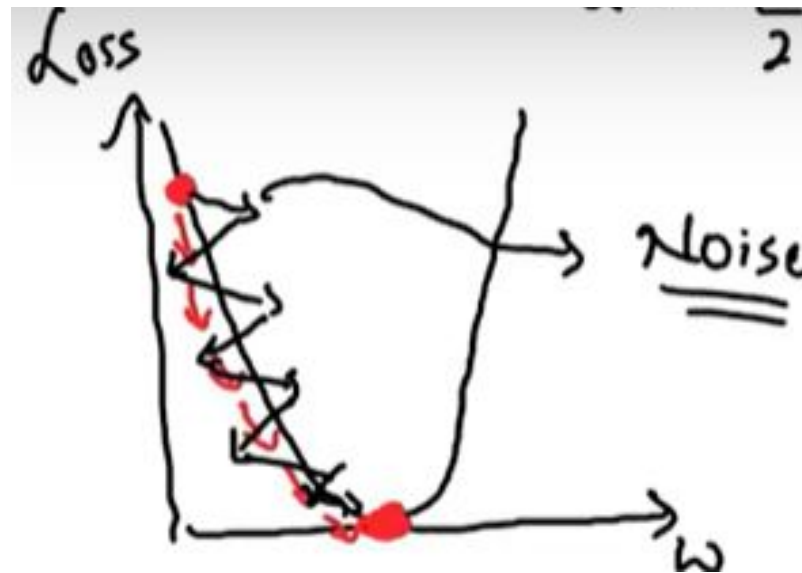
<---- Back Propagation

Iteration 3 : 1000 batch

----> Forward Propagation

<---- Back Propagation

Total iterations = 10 (10k/1000)



Advantages:

The model update frequency is higher than BGD: In MGD, we are not waiting for entire data, we are just passing 50 records or 200 or 100 or 256, then we are passing for optimization.

The batching allows both efficiency of not having all training data in memory and algorithms implementations. We are controlling memory consumption as well to store losses for each and every datasets.

The batches updates provide a computationally more efficient process than SGD.

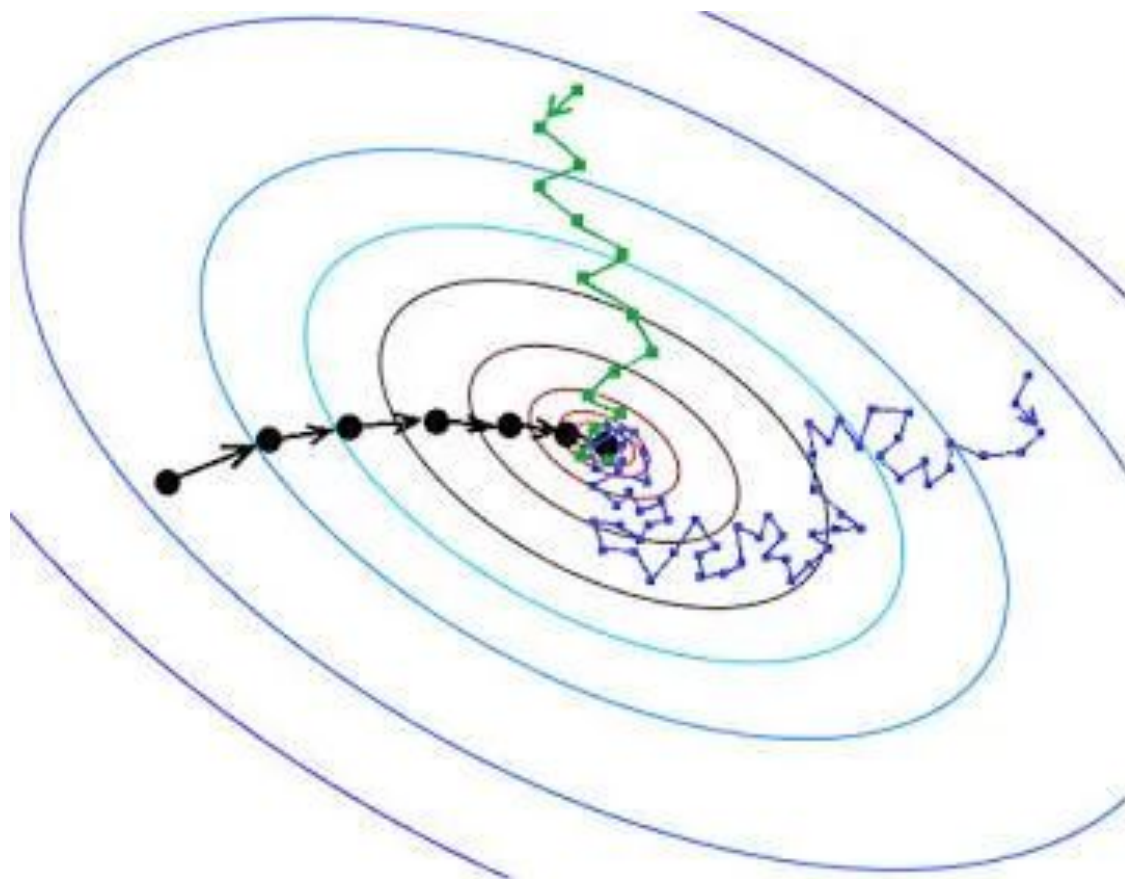
Disadvantages:

No guarantee of convergence of a error in a better way.

Since the 50 sample records we take , are not representing the properties (or variance) of entire datasets. Do, this is the reason that we will not be able to get an convergence i.e., we won't get absolute global or local minima at any point of a time.

While using MGD, since we are taking records in batches, so, it might happen that in some batches, we get some error and in dome other batches, we get some other error. So, we will have to control the learning rate by ourself , whenever we use MGD. If learning rate is very low, so the convergence rate will also fall. If learning rate is too high, we won't get an absolute global or local minima. So we need to control the learning rate.

If the batch size = total no. of data, then in this case, BGD = MGD..



Batch GD

- Slowest
- Perfect gradient

Stochastic GD

- Fastest
- Rough-estimate grad

Mini-batch GD

- Compromise

For every epoch : No of iterations

BGD

Iteration = 1 (whole 10k records)

Epoch ----->
-----<

SGD

Iteration = No of Records but 1 record at a time

Epoch ----->
-----<

Mini batch SGD

Iteration = $10k / \text{batch size}$

Epoch ----->
-----<

SDG with Momentum

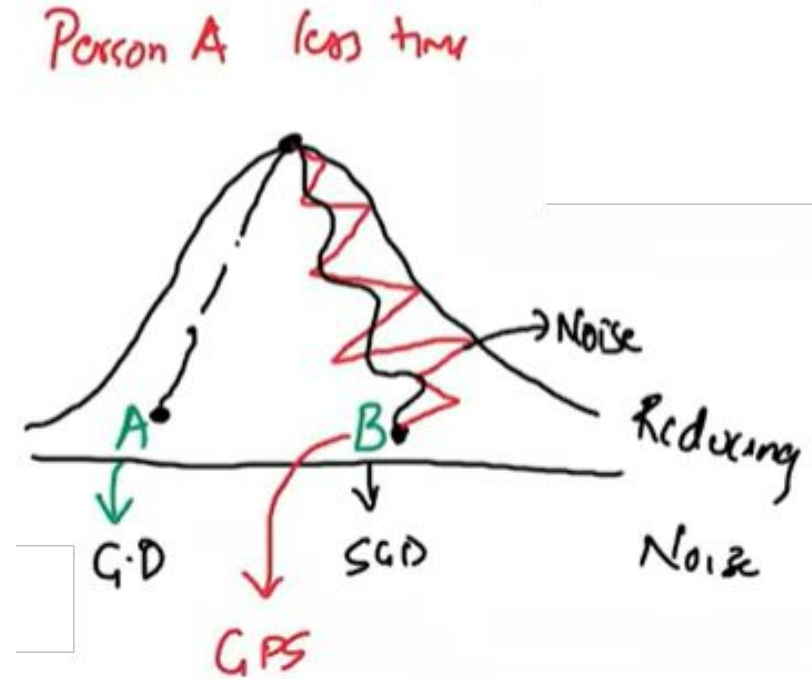
$$\mathbf{w}_{t+1} = \mathbf{w}_t + \mathbf{v}_{t+1}$$

Velocity

$$\mathbf{v}_{t+1} = \rho \mathbf{v}_t - \eta \mathbf{g}(\mathbf{w}_t)$$

Momentum

Update rule



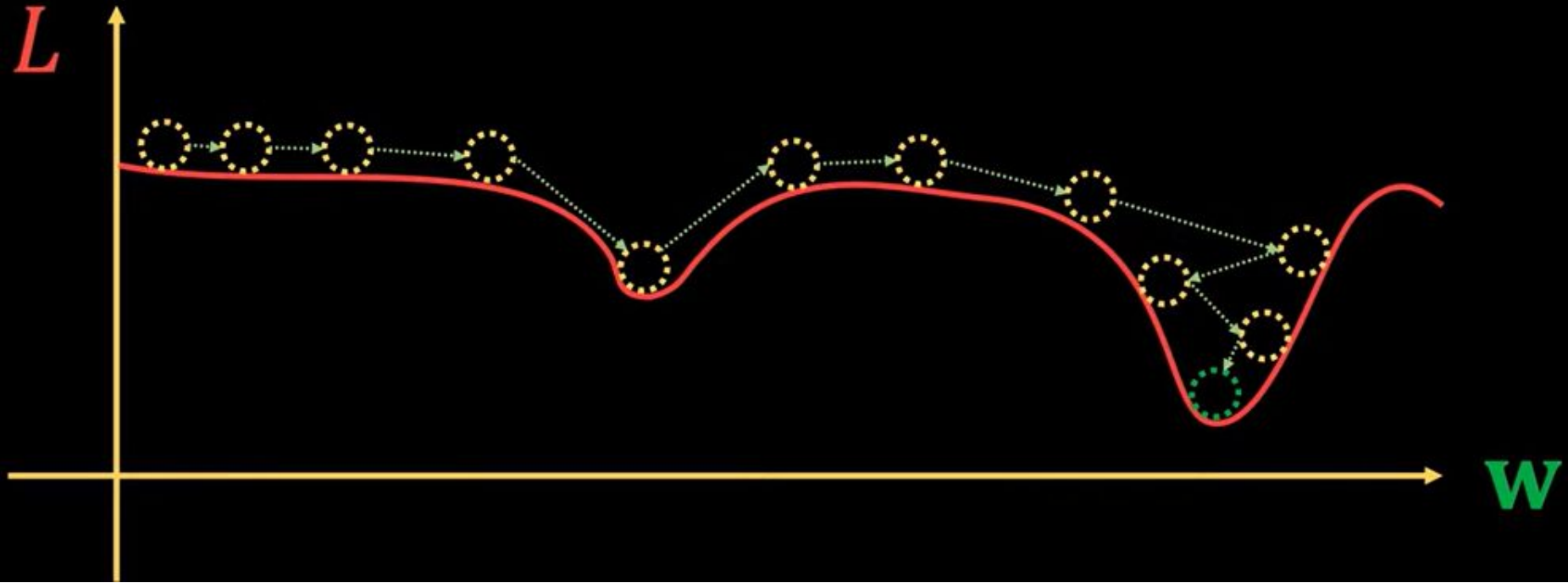
Method of Speeding up the convergence of iteration methods

SDG with Momentum

$$\mathbf{w}_{t+1} = \mathbf{w}_t + \underbrace{\rho \mathbf{v}_t}_{\text{Velocity jump}} - \underbrace{\eta \mathbf{g}(\mathbf{w}_t)}_{\text{Gradient jump}}$$

Update rule

Low learning rate



Summary of Gradient Descent

Gradient descent is an optimization algorithm used to minimize a loss function by iteratively moving in the direction of the steepest descent as defined by the negative of the gradient.

We can use it in various settings, such as linear regression and neural networks.

Problems with Gradient Descent

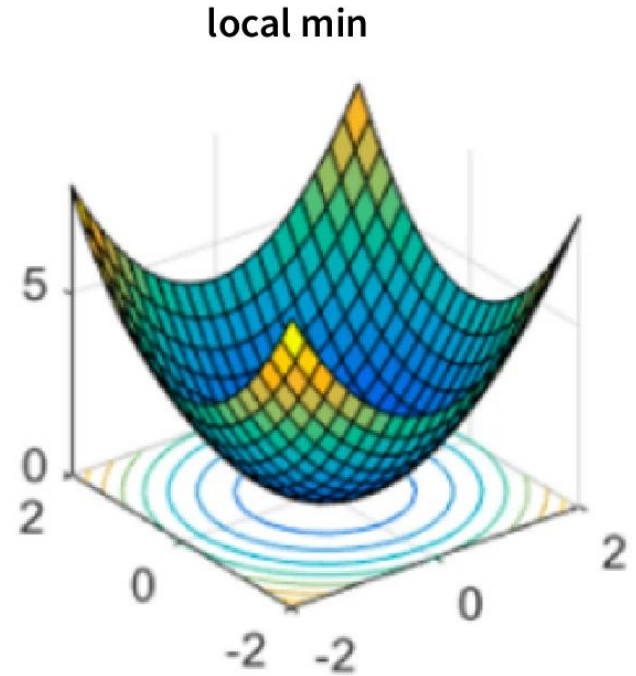
Gradient descent is an optimization algorithm used to minimize a loss function by iteratively moving in the direction of the steepest descent as defined by the negative of the gradient.

We can use it in various settings, such as linear regression and neural networks.

Local Minima

Gradient descent can get stuck in local minima, points that are not the global minimum of the cost function but are still lower than the surrounding points.

This can occur when the cost function has multiple valleys, and the algorithm gets stuck in one instead of reaching the global minimum.

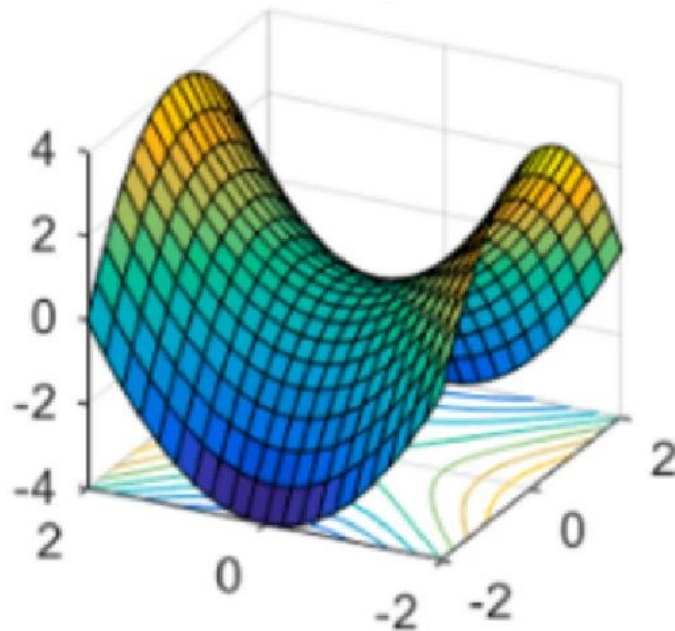


Saddle Points

A saddle point is a point in the cost function where one dimension has a higher value than the surrounding points, and the other has a lower value.

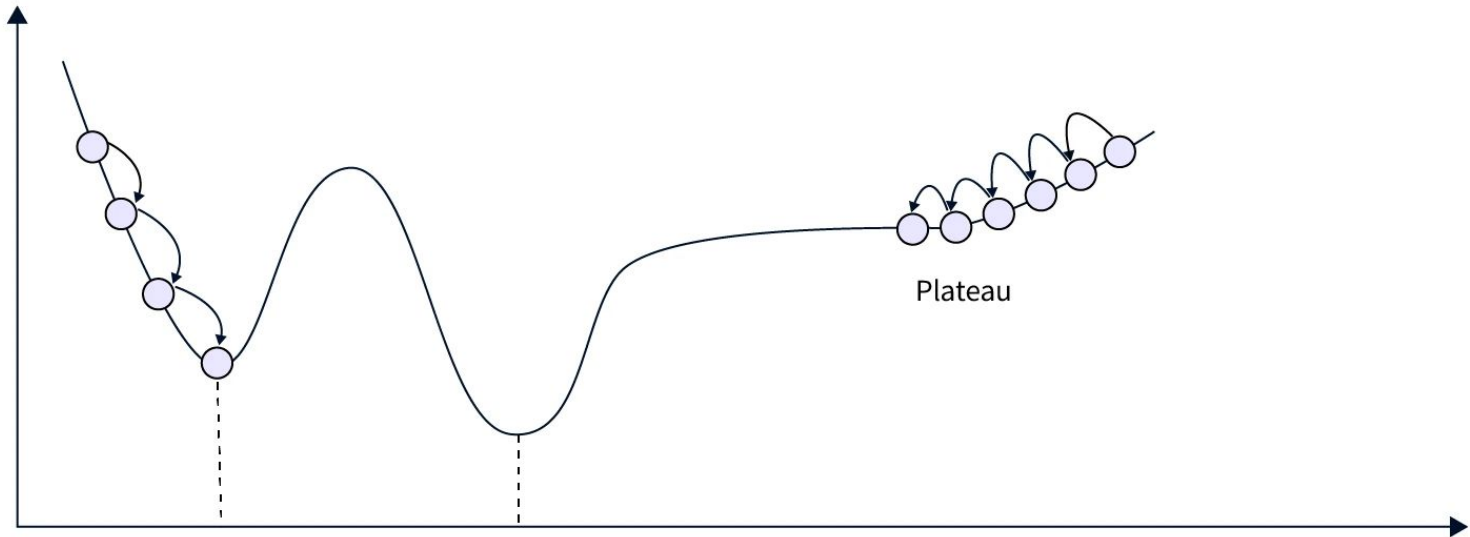
Gradient descent can get stuck at these points because the gradients in one direction point towards a lower value, while those in the other direction point towards a higher value.

Saddle Point



Plateaus

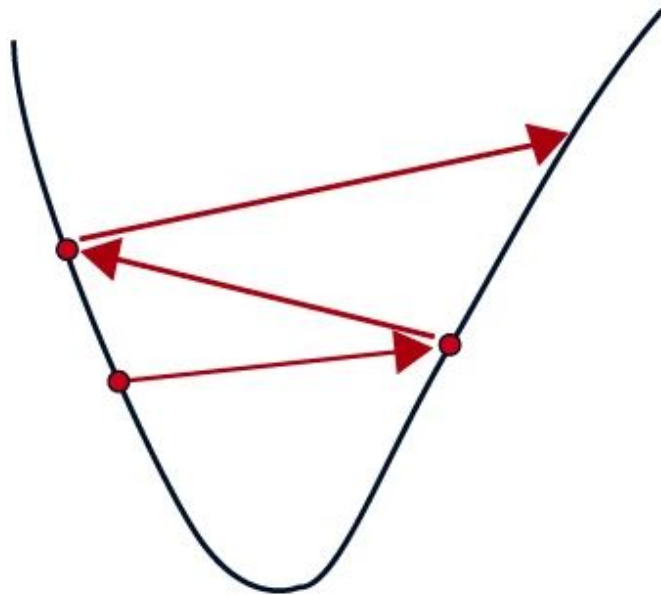
A plateau is a region in the cost function where the gradients are very small or close to zero. This can cause gradient descent to take a long time or not converge.



Oscillations

Oscillations occur when the learning rate is too high, causing the algorithm to overshoot the minimum and oscillate back and forth.

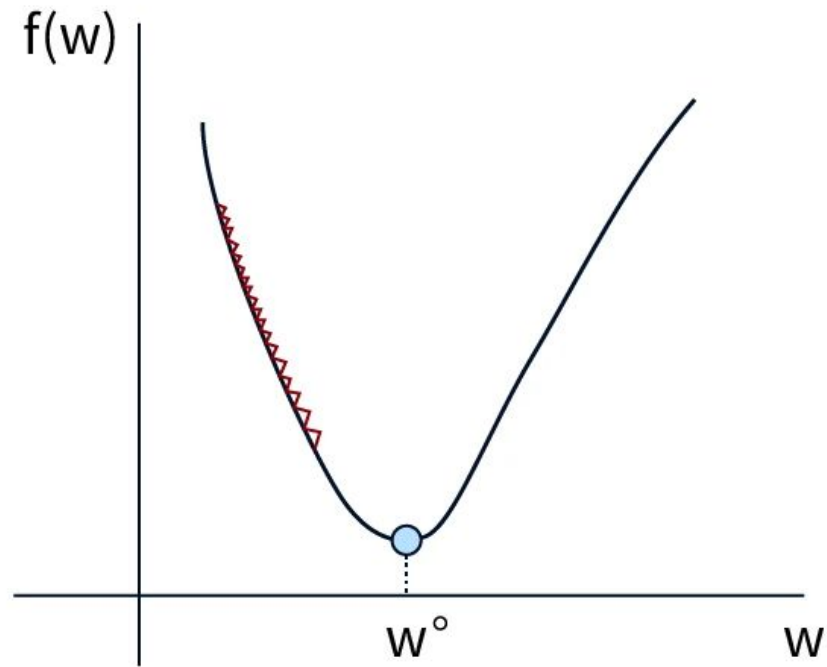
Big Learning Rate



Slow Convergence

Gradient descent can converge very slowly when the cost function is complex or has many local minima.

This means the algorithm may take a long time to find the global minimum.



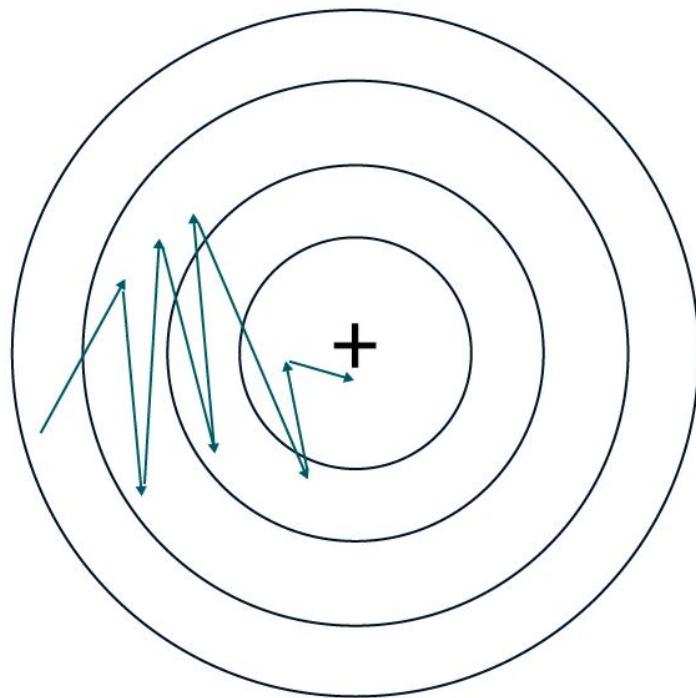
Too Small: converge very slowly

Stochastic

In stochastic gradient descent, the cost function is evaluated at random samples from the data set.

This introduces randomness into the algorithm, making converging to a global minimum more difficult.

Stochastic



Momentum-based Gradient Optimizer

The Momentum-based Gradient Optimizer has several advantages over the basic Gradient Descent algorithm, including **faster convergence, improved stability, and the ability to overcome local minima.**

In the Momentum-based Gradient Optimizer, a fraction of the previous update is added to the current update, which creates a momentum effect that helps the algorithm to move faster towards the minimum.

It is widely used in deep learning applications and is an important optimization technique for training deep neural networks.

How Momentum fix the problems?

Momentum helps to,

- Escape local minima and saddle points
- Aids in faster convergence by reducing oscillations
- Smooths out weight updates for stability
- Reduces model complexity and prevents overfitting
- Can be used in combination with other optimization algorithms for improved performance.

Thank You